



CS240 Algorithm Design and Analysis

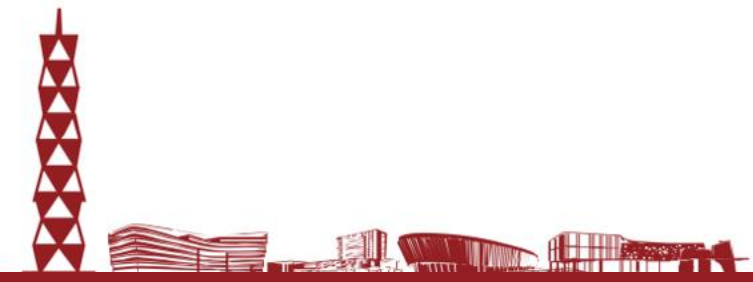
Final Recitation

CS240 TAs

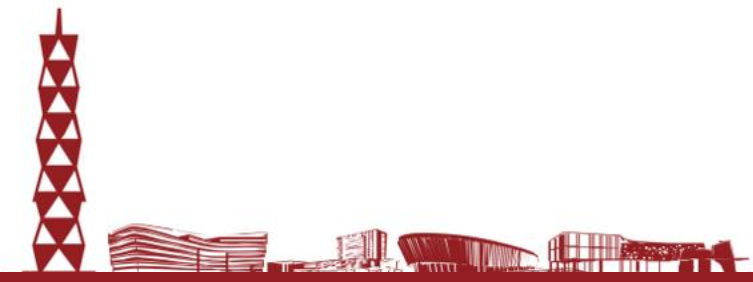
Spring 2026

2026.06.18

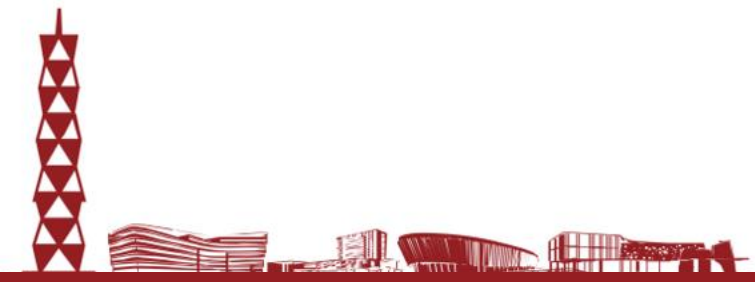
- **June 25th from 10:30 PM to 12:30 PM.** Arrive 10 minutes early!
- **Classroom 201 in the Teaching Center.**
- **Covering: All!**
- **DO NOT CHEAT!**



- Topics covered in this review may not appear in the exam.
- Topics not covered in this review may appear in the exam.

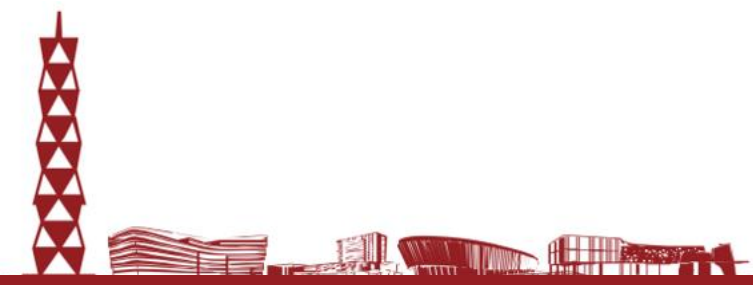


- Before the mid-term
- NP and Computational Intractability
- FPT
- Local Search
- Amortized Analysis
- Randomized Algorithms
- Approximation Algorithms





Before the mid-term





Asymptotic Order of Growth



- **Upper bounds.** $T(n)$ is $O(f(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that for all $n \geq n_0$ we have $T(n) \leq c \cdot f(n)$
- **Lower bounds.** $T(n)$ is $\Omega(f(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that for all $n \geq n_0$ we have $T(n) \geq c \cdot f(n)$
- **Tight bounds.** $T(n)$ is $\Theta(f(n))$ if $T(n)$ is both $O(f(n))$ and $\Omega(f(n))$.
 - Exist constants c_1, c_2, n_0 , such that $c_1 f(n) \leq T(n) \leq c_2 f(n)$ for all $n \geq n_0$
- Let $T(n) = \frac{1}{2}n^2 + 3n$. Which of the following statements are true?
 - $T(n) = O(n)$
 - $T(n) = \Omega(n)$
 - $T(n) = \Theta(n)$
 - $T(n) = O(n^3)$

$$f(n) = \begin{matrix} O(g(n)) & \Omega(g(n)) \\ o(g(n)) & \Theta(g(n)) & \omega(g(n)) \end{matrix}$$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \begin{matrix} \text{---} & \text{---} & \text{---} \\ 0 & 0 < c < \infty & \infty \end{matrix}$$



Core idea

Build the solution step by step and make the best local choice at each stage.

A greedy algorithm never revisits earlier decisions, so the local rule must be safe — not just intuitive.

When greedy is correct, it is usually simpler and faster than dynamic programming or exhaustive search.

When it works

Greedy-choice property: an optimal solution can start with the greedy choice.

Optimal substructure: after one choice, the remaining subproblem is still optimizable.

Typical cost: $O(n \log n)$ after sorting or maintaining a priority queue.

General framework

1. Order the candidates

Sort intervals, deadlines, edges, or pages by a local priority.

2. Test feasibility

Only keep a choice if it stays compatible with the partial solution.

3. Update a small state

Examples: last finish time, current connected components, cache contents.

4. Prove correctness

Use a proof template such as stays-ahead, exchange, or a structural property.

Greedy proofs usually show that the local choice is safe — not just that it looks reasonable.

1. Greedy stays ahead

Compare the greedy partial solution with any other solution after each step.

Show the greedy prefix is never worse, so the final answer is optimal.

Typical link: interval scheduling.

2. Exchange argument

Start from an optimal solution and transform it toward the greedy one.

Each swap keeps the solution feasible and no worse than before.

Typical link: minimizing lateness; also useful in caching proofs.

3. Structural property

Prove each greedy move is forced by a theorem or invariant.

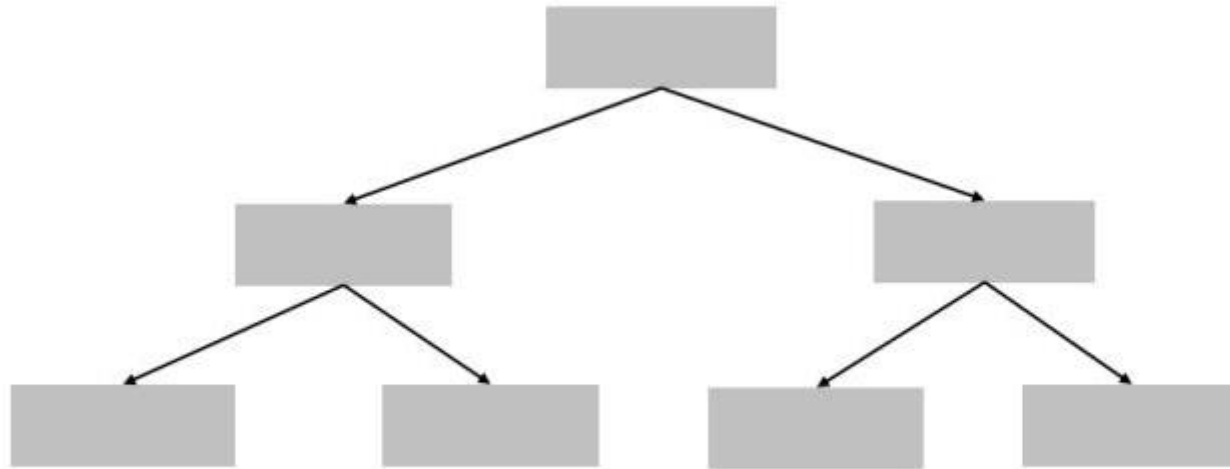
Examples: cut/cycle properties for MST; clustering through the MST structure.

Useful when the problem has a strong graph or matroid-like structure.

Fast exam checklist:

objective → local rule → feasibility condition → counterexample to tempting wrong rules → proof template → complexity

- **Greed.** Build up a solution incrementally, myopically optimizing some local criterion
- **Divide-and-conquer.** Break up a problem into a few sub-problems, solve each sub-problem independently and recursively, and combine solution to sub-problems to form solution to original problem



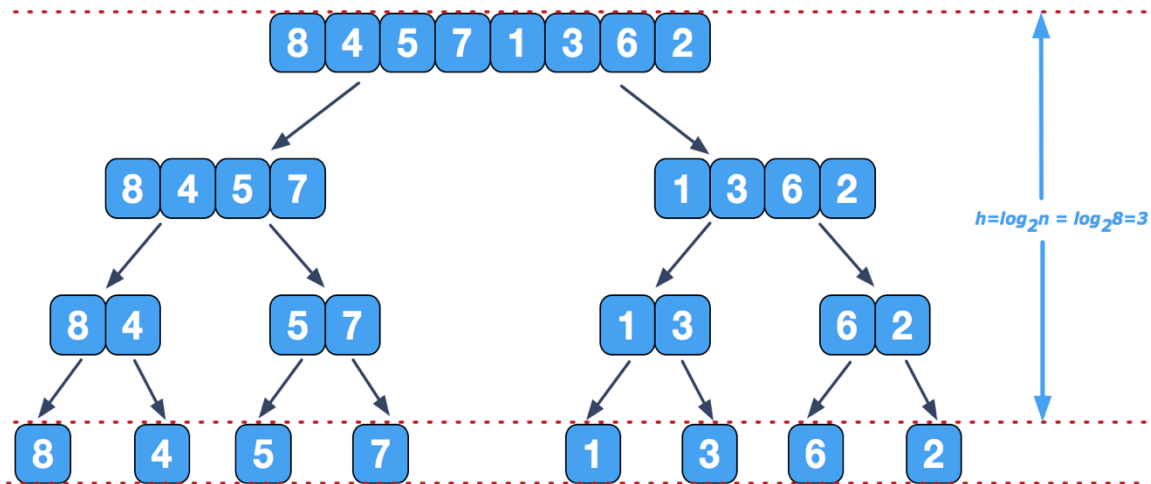


Mergesort



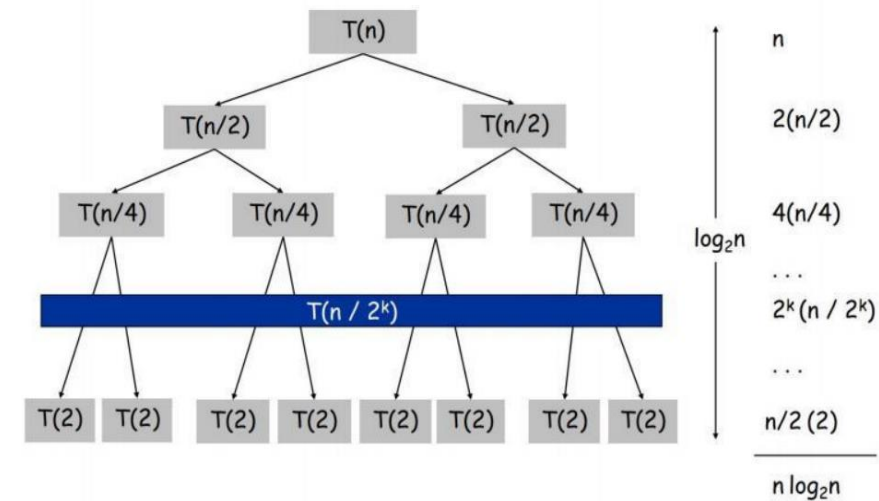
Mergesort

- Divide array into two halves
- Recursively sort each half
- Merge two halves to make sorted whole



Proof by Recursive Tree

$$T(n) = \begin{cases} 0 & \text{if } n = 1 \\ \underbrace{2T(n/2)}_{\text{sorting both halves}} + \underbrace{n}_{\text{merging}} & \text{otherwise} \end{cases}$$



Proof by Induction

- Base case: $n = 1$
- Inductive hypothesis: $T(n) = n \log_2 n$
- Goal: show that $T(2n) = 2n \log_2(2n)$

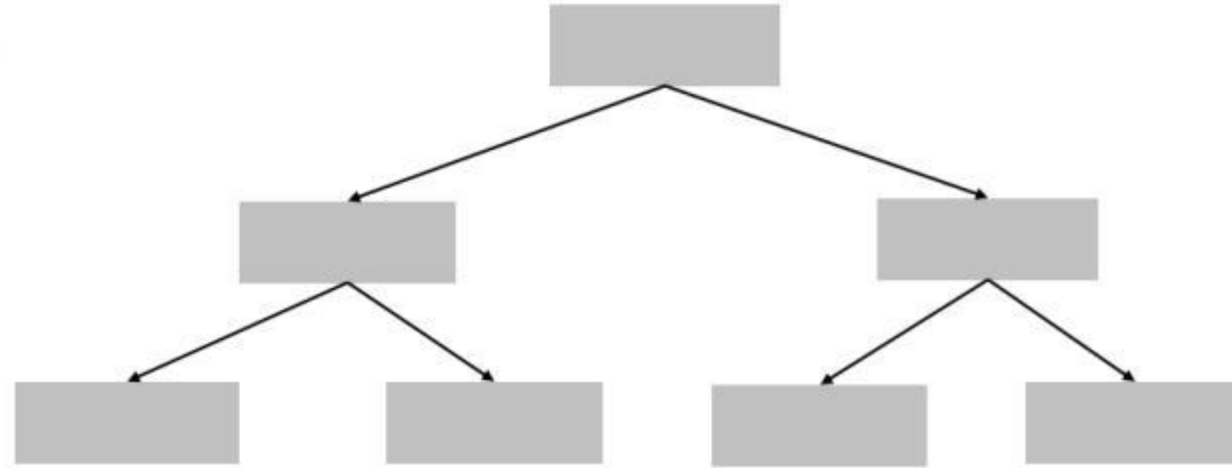
$$\begin{aligned} T(2n) &= 2T(n) + 2n \\ &= 2n \log_2 n + 2n \\ &= 2n(\log_2(2n) - 1) + 2n \\ &= 2n \log_2(2n) \end{aligned}$$



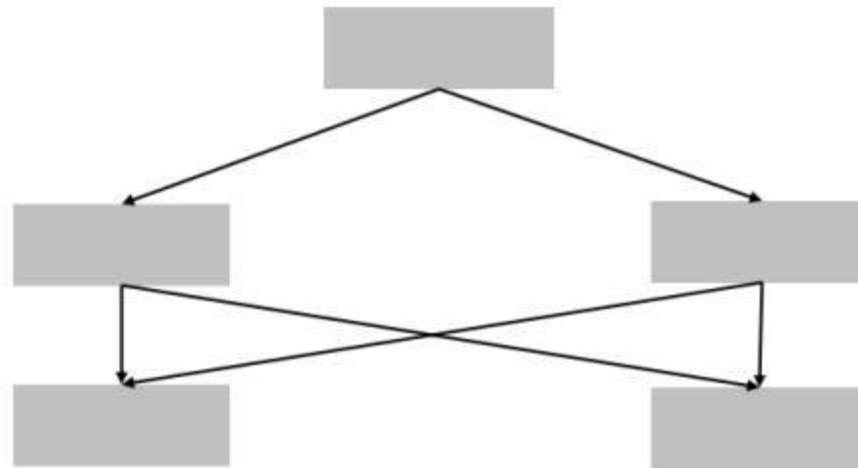
Divide-and-conquer VS. Dynamic Programming



Divide-and-conquer



Dynamic programming





Dynamic Programming

- **Basic idea**

- Polynomial number of sub-problems with a natural ordering from smallest to largest
- Optimal solution to a sub-problem can be constructed from optimal solutions of smaller sub-problems
- Sub-problems are overlapping!

- **Guideline**

- Define the sub-problems
 - $OPT(\dots)$
- Write down the recursive formulas
 - Ex: $OPT(i) = \max(f(OPT(j)), g(OPT(k)), \dots), j, k < i$
- Compute the formulas either bottom-up or top-down



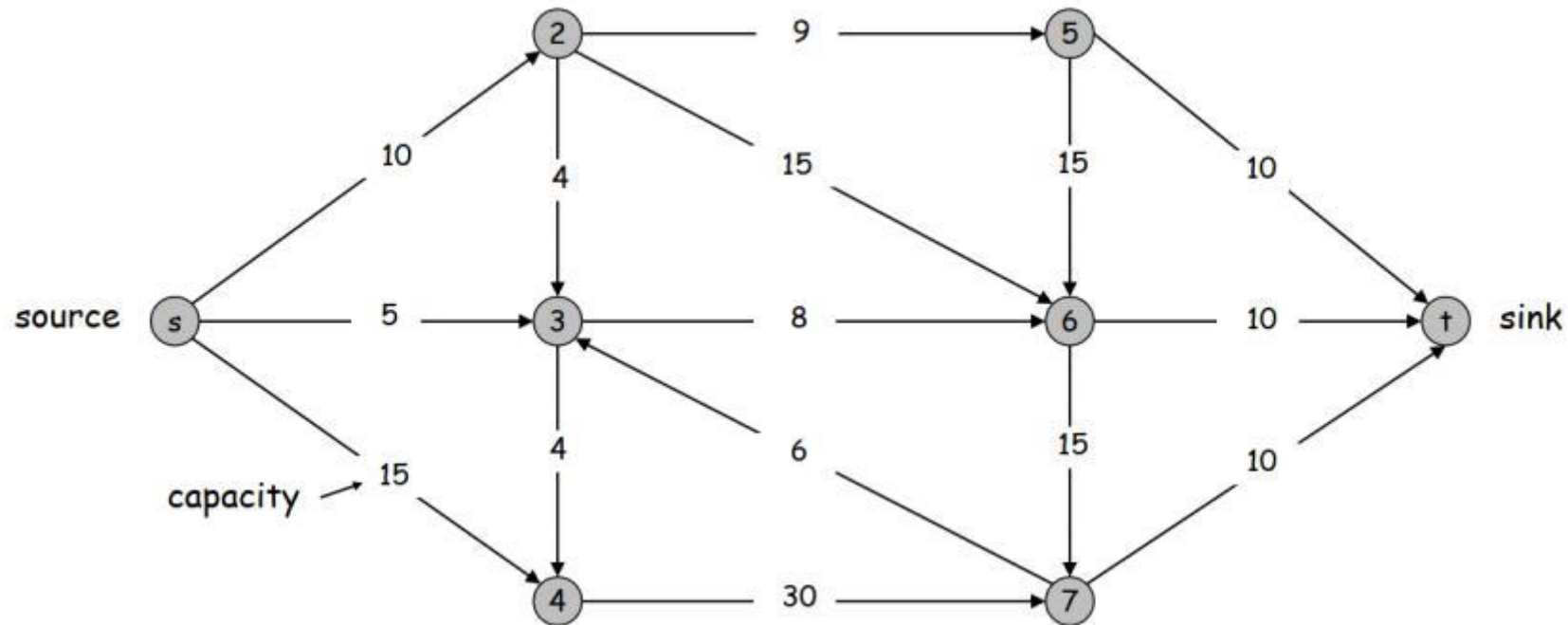


Network Flow



- **Flow network**

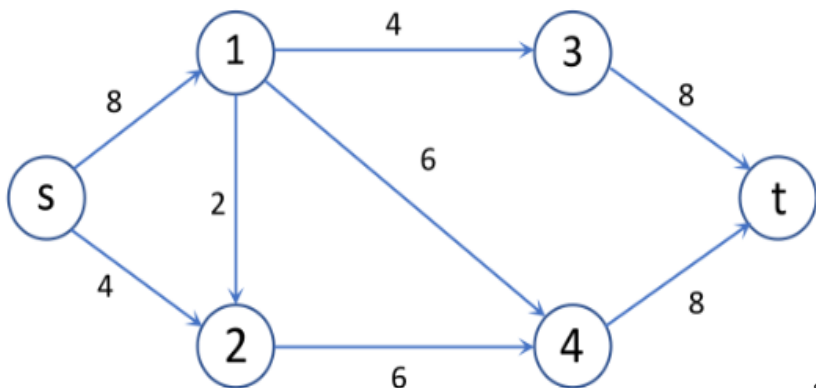
- Abstraction for material **flowing** through the edges
- $G = (V, E)$ = directed graph
- Two distinguished nodes: s = source, t = sink
- $c(e)$ = nonnegative capacity of edge e



Ford-Fulkerson Algorithm

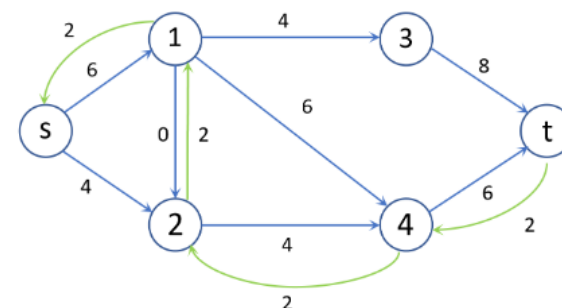
Ford-Fulkerson Algorithm

- Start with $f(e) = 0$ for all edge $e \in E$
- Find an **augmenting path** P in the **residual graph** G_f
- Augment flow along path P
- Repeat until you get stuck

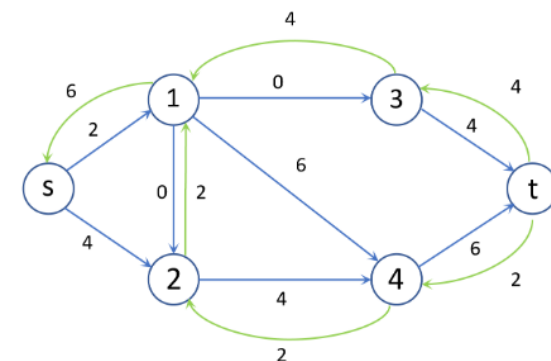


HW2 Problem4: During each iteration, choose the lexicographically smallest available augmenting path.

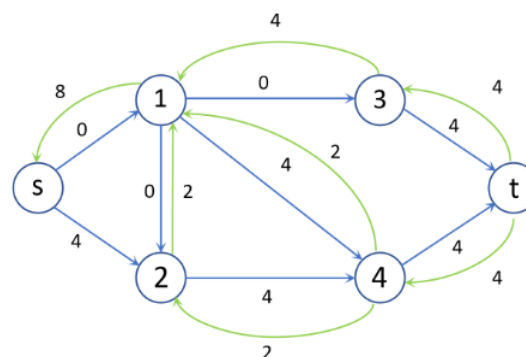
$s \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow t$



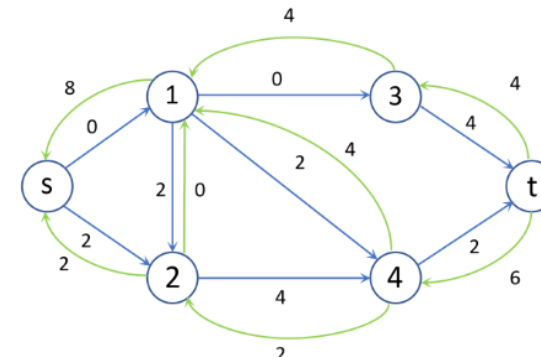
$s \rightarrow 1 \rightarrow 3 \rightarrow t$



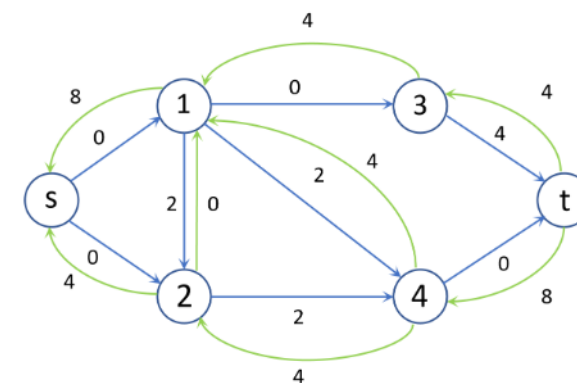
$s \rightarrow 1 \rightarrow 4 \rightarrow t$



$s \rightarrow 2 \rightarrow 1 \rightarrow 4 \rightarrow t$



$s \rightarrow 2 \rightarrow 4 \rightarrow t$





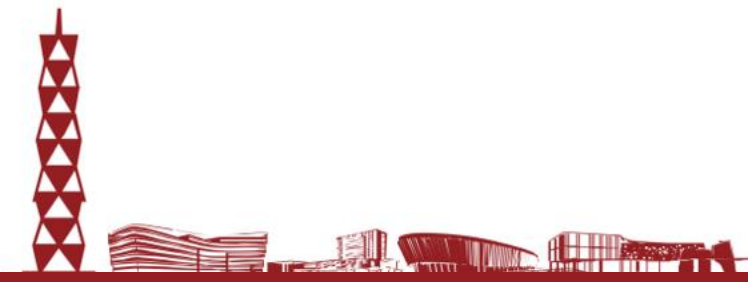
Max-flow and Min-Cut

- **Augmenting path theorem.** Flow f is a max flow if there are no augmenting paths
- **Max-flow min-cut theorem.** The value of the max flow is equal to the value of the min cut
- **Proof strategy.** We prove both simultaneously by showing the equivalence of the following three conditions for any flow f :
 - (i) There exists a cut (A, B) such that $v(f) = \text{cap}(A, B)$
 - (ii) Flow f is a max flow
 - (iii) There is no augmenting path relative to f





NP and Computational Intractability





Key Concepts



- **Decision problem**
 - Answer yes/no
- **P.** Decision problems for which there is a **poly-time** algorithm
- **NP.** Decision problems for which there exists a **poly-time** certifier
 - Algorithm $C(s, t)$ is a **certifier** for problem X if for every string s , $s \in X$ iff there exists a string t such that $C(s, t) = \text{yes}$
- **Co-NP.** Complements of decision problems in NP
- **EXP.** Decision problems for which there is an **exponential-time** algorithm
- **Claim.** $P \subseteq NP, \text{co-NP} \subseteq EXP$





NP-Completeness & NP-Hard



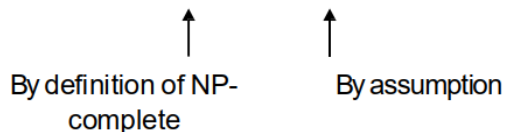
- **NP-hard.** [Bell Labs, Steve Cook, Ron Rivest, Sartaj Sahni] A problem such that every problem in NP reduces to it

$$\forall X \in NP, X \leq_p Y$$

- **NP-complete.** A problem in NP such that every problem in NP polynomial reduces to it

$$NP - complete = NP \cap NP\text{-hard}$$

- **Remark.** Once we establish first “natural” NP-complete problem, others fall like dominoes
- **Recipe to establish NP-completeness of problem Y**
 - Step 1. Show that Y is in NP
 - Step 2. Choose an NP-complete problem X
 - Step 3. Prove that $X \leq_p Y$
- **Theorem.** If X is NP-complete problem and Y is a problem in NP with the property that $X \leq_p Y$, then Y is NP- complete
- **Pf.** Let W be any problem in NP. Then $W \leq_p X \leq_p Y$
 - By transitivity, $W \leq_p Y$
 - Hence Y is NP-complete





Polynomial-Time Reduction

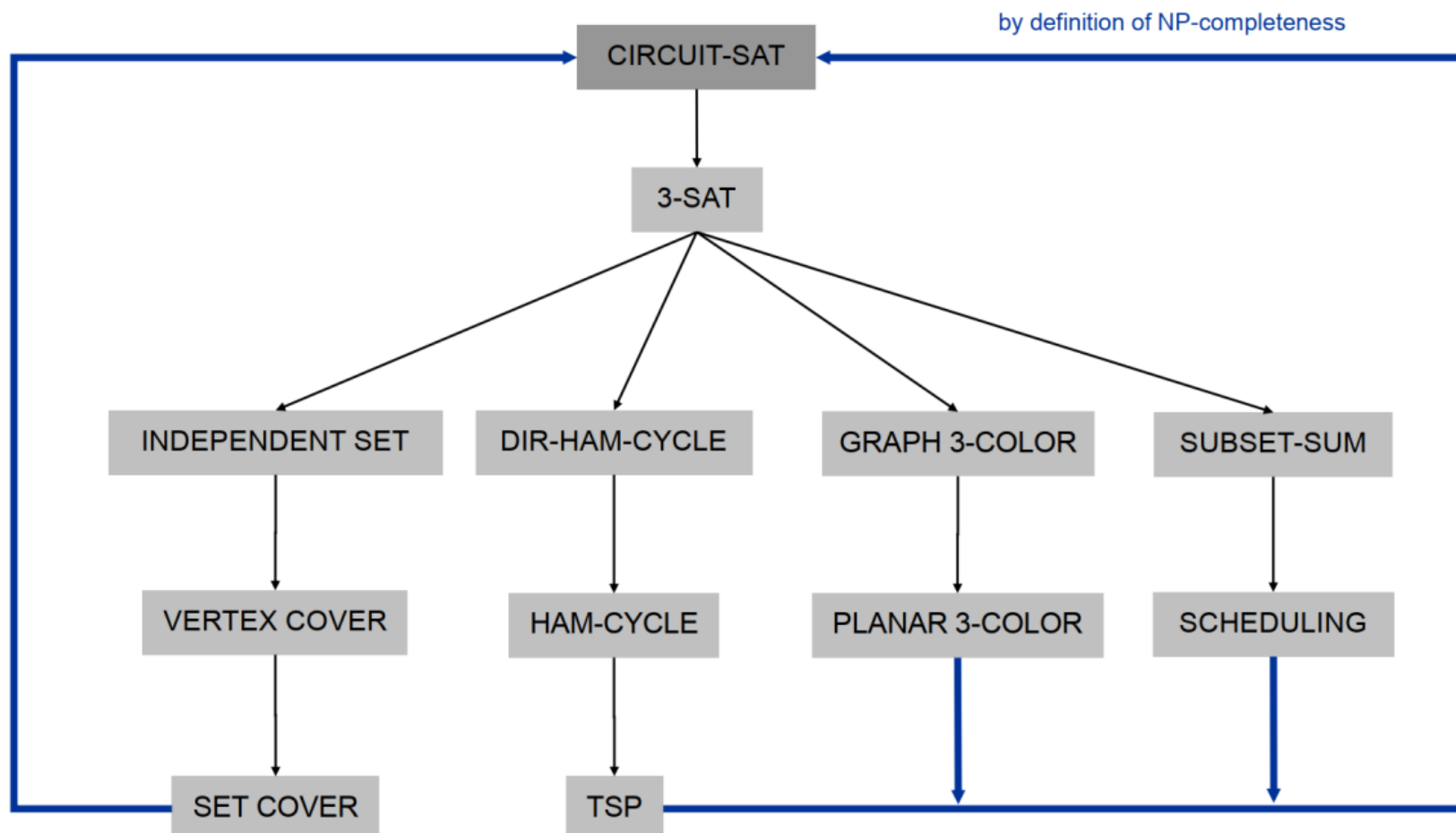


- **Reduction.** Problem X **polynomial-time reduces** to problem Y if arbitrary instances of problem X can be solved using:
 - Polynomial number of standard computational steps, plus
 - Polynomial number of calls to oracle that solves problem Y
- **Notation.** $X \leq_p Y$
- **Common approach: polynomial transformation**
 - Given any input x to X , **construct** an input y in poly-time such that x is a yes instance of X **iff** y is a yes instance of Y



NP-Completeness

- **Observation.** All problems below are NP-complete and polynomial reduce to one another!





NP-Completeness Prove



Notice: When proving that a problem A is NP-complete, you need to strictly follow the steps below and explicitly state each part.

- Show that $A \in \text{NP}$.

That is, given a candidate solution, we can verify it in polynomial time.

- Choose a known NP-complete problem B .
- Give a polynomial-time reduction from B to A .

For any instance b of B , construct an instance a of A in polynomial time.

- Prove correctness of the reduction.

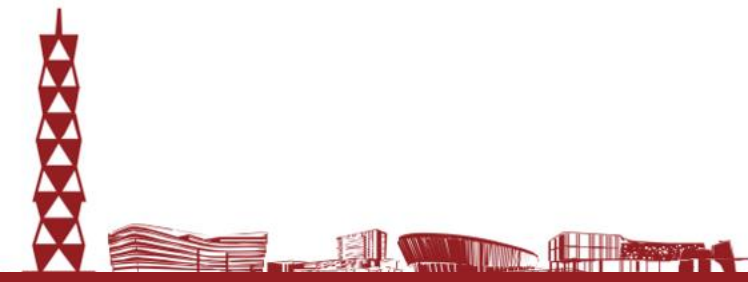
Show that

$$b \text{ is a YES-instance of } B \iff a \text{ is a YES-instance of } A.$$





Fixed-Parameter Tractable (FPT)

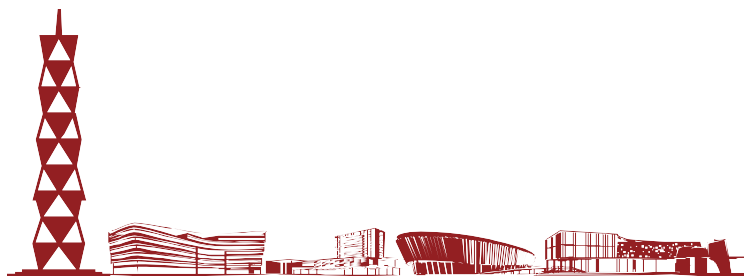




Why FPT



- **Q.** Suppose I need to solve an NP-complete problem. What should I do?
- **A.** Theory says you're unlikely to find poly-time algorithm.
- **Must sacrifice at least one of three desired features.**
 - Solve problem in polynomial time.
 - Solve problem to optimality.
 - Solve **arbitrary instances** of the problem.
- **This lecture.** Solve some special cases of NP-complete problems that arise in practice.

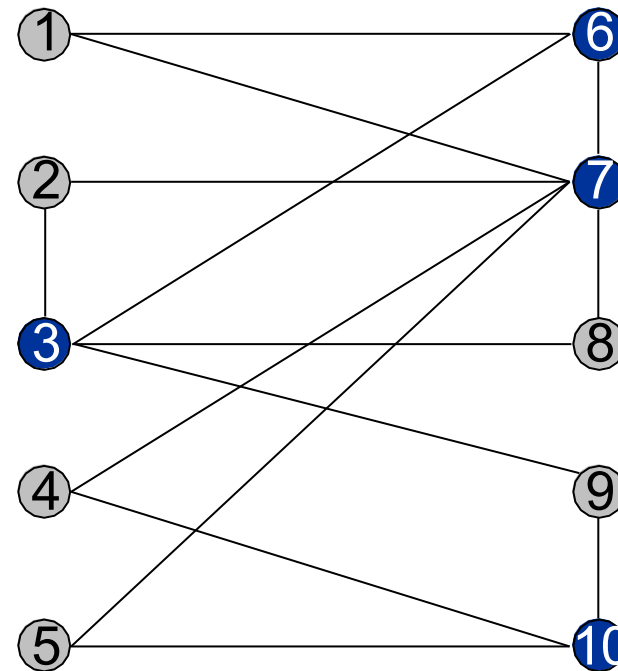




Vertex Cover

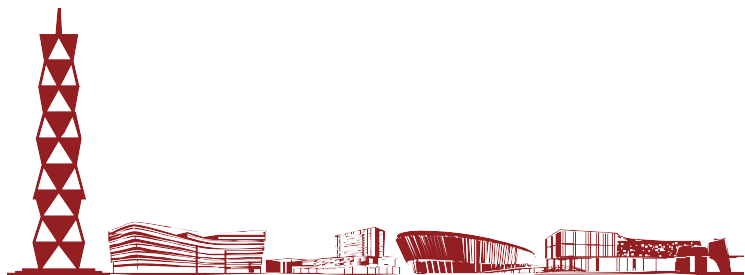


- **Problem definition:** Given a graph $G = (V, E)$ and an integer k , is there a subset of vertices $S \subseteq V$ such that $|S| \leq k$, and for each edge (u, v) , either $u \in S$, or $v \in S$, or both



$$k = 4$$

$$S = \{ 3, 6, 7, 10 \}$$





Finding Small Vertex Covers



- **Q.** What if k is small?
- **Brute force.** $O(kn^{k+1})$.
 - Try all $C(n, k) = O(n^k)$ subsets of size k .
 - Takes $O(kn)$ time to check whether a subset is a vertex cover.
- **Goal.** Limit exponential dependency on k , e.g., to $O(2^k n)$.
- **Ex.** $n = 1,000$, $k = 10$.
 - **Brute force:** $kn^{k+1} = 10^{34} \Rightarrow$ infeasible.
 - **Better:** $2^k n = 10^6 \Rightarrow$ feasible.
- **Remark.** If k is a constant, then the algorithm is poly-time
- If k is a small constant, then it's also practical
- **Parameterized complexity.** FPT (fixed parameter tractable) with respect to some parameter k is the class of problems solvable in time $f(k) \cdot \text{poly}(n)$





Wavelength-Division Multiplexing



Background. More than one communication links can share the same portion of a fiber optic cable, provided they are transmitted using different wavelengths.

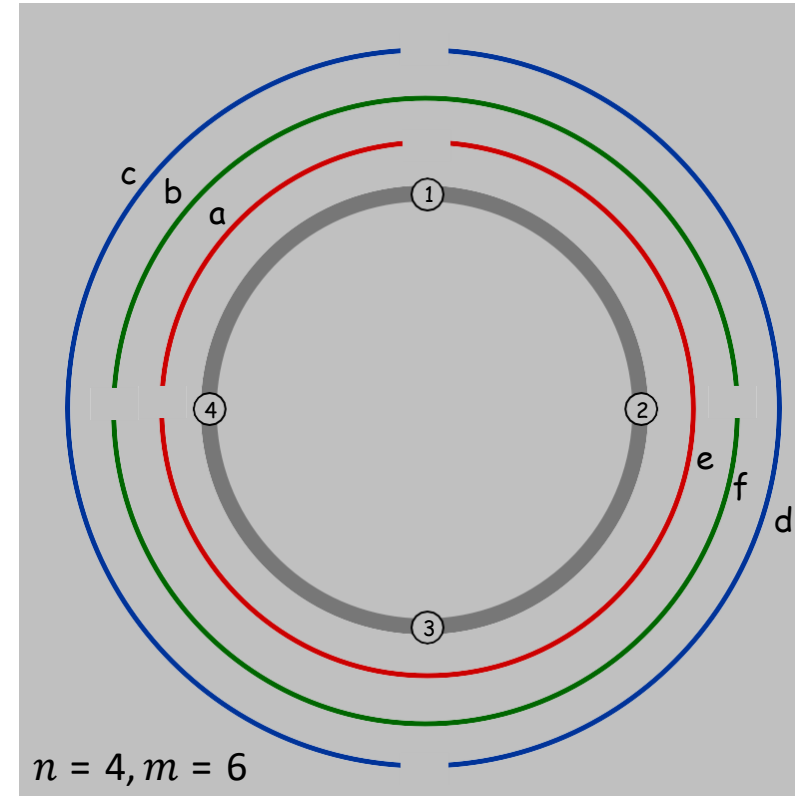
Problem definition. Given a graph $G = (V, E)$, and m paths $p_1, p_2, \dots, p_m$, assign each path a color so that any two paths that share an edge must have different colors. The goal is to use as few colors as possible.

Ring topology. Consider the special case is when graph is a **cycle** on n nodes.

Bad news. NP-complete, even on rings.

Brute force. Can determine if k colors suffice in $O(k^m)$ time by trying all k -colorings.

Goal. $f(k) \cdot \text{poly}(m, n)$ time.





Circular Arc Coloring: Running Time

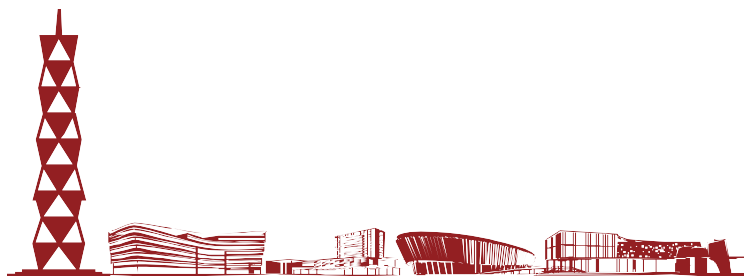


Running time. $O(k! \cdot n)$

- n phases of the algorithm.
- Bottleneck in each phase is enumerating all consistent colorings.
- There are at most k intervals through v_i , so there are at most $k!$ colorings to consider.

Remark. This is $\text{poly}(n)$ time if $k = O(\log n / \log \log n)$

This algorithm is practical for small values of k (say $k = 10$) even if the number of nodes n (or paths) is large





Local Search



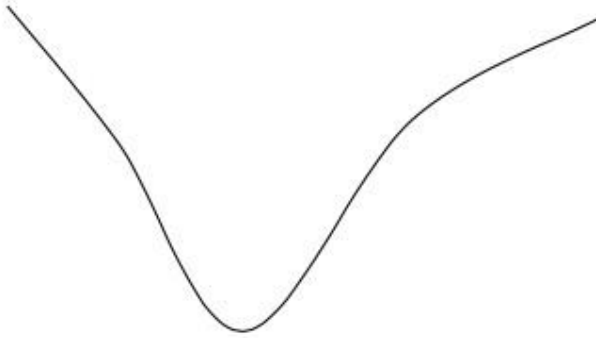


Local Search

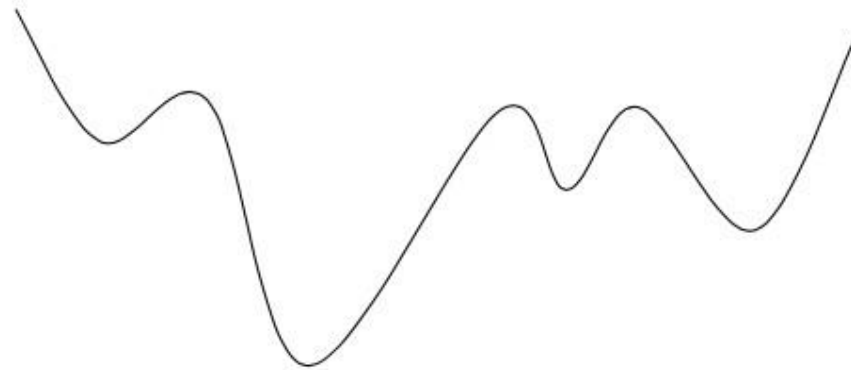
Local search. Algorithm that explores the space of possible solutions in sequential fashion, moving from a current solution to a "nearby" one.

Neighbor relation. Let $S \sim S'$ be a neighbor relation for the problem.

Gradient descent. Let S denote current solution. If there is a neighbor S' of S with strictly lower cost, replace S with the neighbor whose cost is as small as possible. Otherwise, terminate the algorithm.



A funnel



A jagged funnel



Maximum Cut

Maximum cut. Given an undirected graph $G = (V, E)$ with positive integer edge weights w_e , find a node partition (A, B) such that the total weight of edges crossing the cut is maximized.

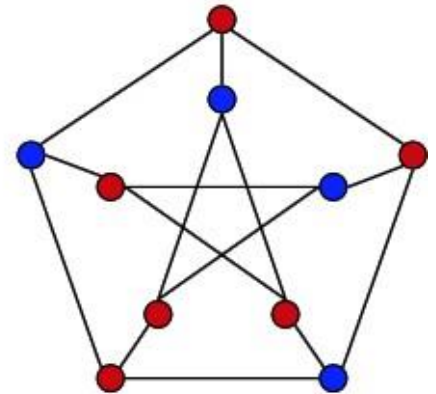
$$w(A, B) := \sum_{u \in A, v \in B} w_{uv}$$

Note:

- The Min-Cut problem can be solved in poly time.
- The Max-Cut problem is NP-hard.

Toy application

- n activities, m people
 - Each person wants to participate in two of the activities
 - Schedule each activity in the morning or afternoon to maximize number of people that can enjoy both activities
- **Real applications.** Circuit layout, statistical physics





Maximum Cut

Single-flip neighborhood. Given a partition (A, B), move one node from A to B, or one from B to A if it improves the solution.

Local search algorithm.

```
Max-Cut-Local (G, w) {  
    Pick a random node partition (A, B)  
  
    while ( $\exists$  improving node v) {  
        if (v is in A) move v to B  
        else           move v to A  
    }  
  
    return (A, B)  
}
```




Maximum Cut: Big Improvement Flips

Local search. Within a factor of 2 for MAX-CUT, but not poly-time!

Big-improvement-flip algorithm. Only choose a node which, when flipped, increases the cut value by at least $\frac{2\varepsilon}{n} w(A, B)$

Claim. Upon termination, big-improvement-flip algorithm returns a cut (A, B) with $(2 + \varepsilon) w(A, B) \geq w(A^*, B^*)$.

Pf idea. Add $\frac{2\varepsilon}{n} w(A, B)$ to each inequality in original proof.

Claim. Big-improvement-flip algorithm terminates after $O(\varepsilon^{-1} n \log W)$ flips, where $W = \sum_e w_e$.

- Each flip improves cut value by at least a factor of $(1 + \varepsilon/n)$.
- After n/ε iterations the cut value improves by a factor of 2
- Cut value can be doubled at most $\log_2 W$ times.

if $x \geq 1$, $(1+1/x)^x \geq 2$



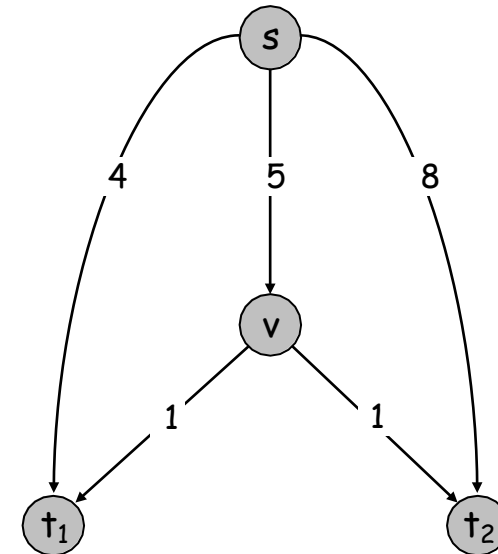
Multicast Routing with Fair Cost Sharing

Multicast routing. Given a directed graph $G = (V, E)$ with edge costs $c_e \geq 0$, a source node s , and k agents located at terminal nodes $t_1, \dots, t_k$.

Agent j must construct a path P_j from node s to its terminal t_j .

Fair share. If x agents use edge e , they each pay c_e / x .

1	2	1 pays	2 pays
outer	outer	4	8
outer	middle	4	$5 + 1$
middle	outer	$5 + 1$	8
middle	middle	$5/2 + 1$	$5/2 + 1$





Nash Equilibrium & Social Optimum

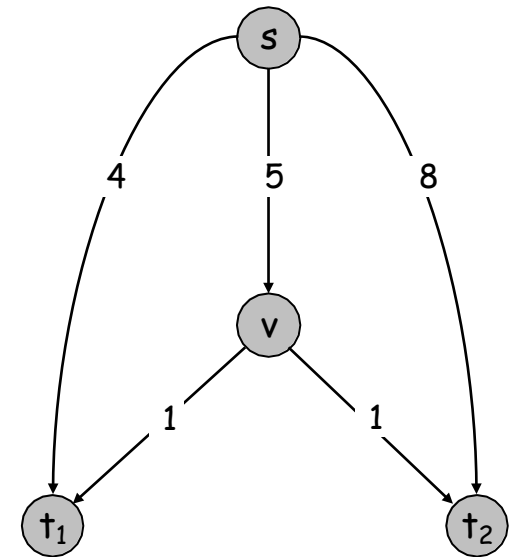
Best response dynamics. Each agent is continually prepared to improve its solution in response to changes made by other agents.

Nash equilibrium. Solution where no agent has an incentive to switch.

Note. Best response dynamics may not terminate since no single objective function is being optimized.

Social optimum. Minimizes total cost to all agents.

Observation. In general, there can be many Nash equilibria. Even when it's unique, it does not necessarily equal the social optimum.





- **Amortized Analysis**
- **Randomized Algorithms**
- **Approximation Algorithms**



CS240 Algorithm HW4 Review

Problem Set 4 as the main review path

amortized analysis

randomized algorithms

P1

**Lazy Disk-
Write
Queue**

P2

**Triangular
Calibration**

P3

**Tombstone
Compaction**

P4

**Random
Audit**

P5

**Random-Priority
Conflict Filter**

answer protocols

Goal: convert each problem into a reusable proof template.

HW4 Proof Map

Five problems, two lenses: amortized sequence bounds and randomized expectation / probability bounds.

Amortized sequence problems

- P1 Queue: each inserted entry is removed once
- P2 Calibration: total triggered cost is triangular
- P3 Compaction: tombstones pay for cleanup

show $\sum \text{actual cost} \leq \sum \text{amortized charge} = O(m)$

- Use aggregate for total-cost counting
- Use accounting / potential when credit is explicit

Randomized one-shot problems

- P4 Audit: count violations found in a sample
- P5 Priority filter: selected vertices are local minima

define indicators $\rightarrow$ compute local Pr $\rightarrow$ sum by linearity

- For failure probability, write the no-success event first
- For expectation, independence is often not needed

P1 · Lazy Disk-Write Queue

Main idea: stale duplicate entries are expensive only once, because every queue entry was previously inserted.

Bad call construction

- 1 **Modify(i) repeated k times**
- 2 **FlushOne writes page i**
- 3 **next FlushOne discards $\Theta(k)$ stale entries**

One FlushOne can cost $\Theta(m)$, but the sequence is still linear.

Aggregate proof

- Each queue insertion is caused by one Modify.
- Each queue entry is deleted at most once.
- Duplicates do not hurt: they are separate inserted entries.

$$\text{total queue work} = O(\text{\#insertions} + \text{\#deletions}) = O(m)$$

Potential-method protocol

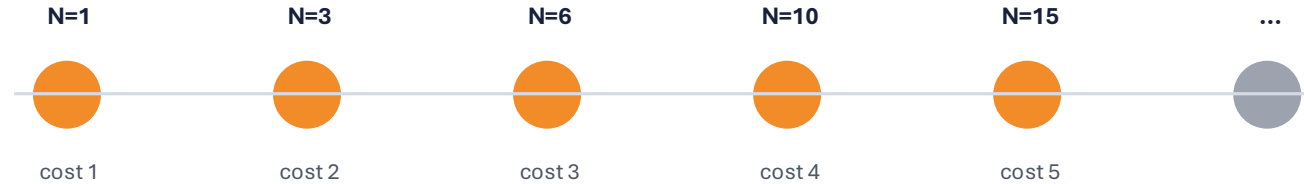
$$\Phi = c \cdot |Q|$$

- Modify: actual $O(1)$, Φ increases by $O(1)$ $\rightarrow$ amortized $O(1)$.
- FlushOne: every removed entry decreases Φ ; the drop pays for stale deletions.
- Choose constant c large enough to cover deletion + dirty check + write-back constants.

P2 · Triangular Calibration Monitor

Main idea: calibration costs are not arbitrary bursts; their total is itself a triangular number.

Trigger timeline



Compute number of calibrations

$$T_q = \frac{q(q+1)}{2} \leq n$$

$$q = \left\lfloor \frac{\sqrt{8n+1} - 1}{2} \right\rfloor$$

- First q calibrations have total cost $1+2+\dots+q = T_q \leq n$.
- Base insertion cost is n . Total cost $\leq 2n$.

Accounting protocol

charge 2 per InsertSample

- Use 1 unit for the base insertion.
- Save 1 unit as calibration credit.
- By any time n , saved credit n covers total calibration cost $\leq n$.

P3 · Archive with Tombstone Compaction

Main idea: tombstones are stored credit; when they outnumber live articles, they pay for cleanup.

Trigger condition



After Remove: $T > L \Rightarrow \text{Compact}()$

Worst case

- A single Remove can trigger Compact.
- Compact scans the whole array and copies all live articles.
- Thus one external operation may cost $\Theta(n)$.

Potential

$$\Phi = 3T$$

- Add: T unchanged $\rightarrow O(1)$.
- Remove without compaction: T increases by 1 $\rightarrow O(1)$.
- Remove with compaction: Φ drops to 0.

Why $3T$ pays

$$\text{cost} \leq \text{scan}(L + T) + \text{copy}(L) = T + 2L < 3T$$

- At trigger time $T > L$.
- Compaction discards all tombstones, so potential release pays for cleanup.

P4 · Random Audit without Replacement

Main idea: use indicators for expectation and a no-hit event for the failure probability.

Expectation by indicators

$$X = \#\{\text{selected violations}\}$$

$$\mathbb{E}[X] = s \cdot \frac{r}{n}$$

- Each selected position has violation probability r/n .
- Linearity gives the expectation even without replacement.

No-violation probability

$$\Pr[X=0] = \frac{\binom{n-r}{s}}{\binom{n}{s}}$$

$$= \prod_{i=0}^{s-1} \frac{n-r-i}{n-i}$$

- If $s > n-r$, this probability is 0.
- This is the exact without-replacement formula.

Exponential bound

$$\Pr[X=0] \leq \left(1 - \frac{r}{n}\right)^s \leq e^{-sr/n}$$

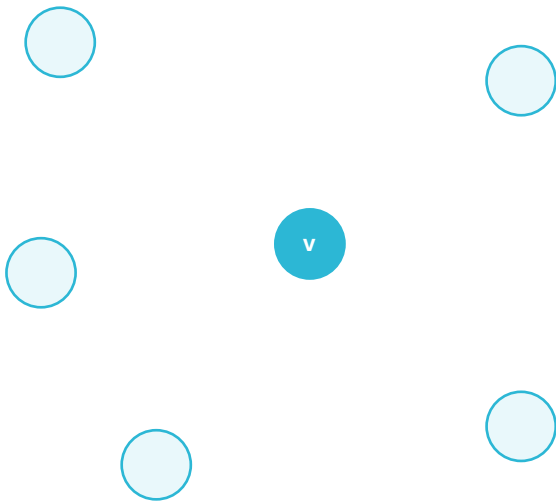
$$s \geq \frac{n}{r} \ln \frac{1}{\delta}$$

sufficient for success
probability $\geq 1-\delta$

P5 · One-Round Random-Priority Conflict Filter

Main idea: selected vertices are local minima in their closed neighborhoods.

Local-minimum rule



v is selected $\Leftrightarrow p(v)$ is smallest among v and all neighbors

Feasibility

- If u and v are adjacent, they cannot both have smaller priority than the other.
- Therefore the selected set S is always independent.

S is an independent set

Expectation

$$\Pr[v \text{ selected}] = \frac{1}{\deg(v) + 1}$$

$$\mathbb{E}[|S|] = \sum_{v \in V} \frac{1}{\deg(v) + 1}$$

$$d\text{-regular: } \mathbb{E}[|S|] = \frac{|V|}{d+1}$$

Continuous priorities imply no ties with probability 1.

Final Protocol Sheet

Use this slide as the grading and solution-writing checklist for HW4.

Amortized sequence

- Show a single call can be large if asked.
- Count the objects/events that cause large work.
- Argue each such object is created and consumed $O(1)$ times.
- Conclude total cost $O(m)$ or $O(n)$.

Potential / accounting

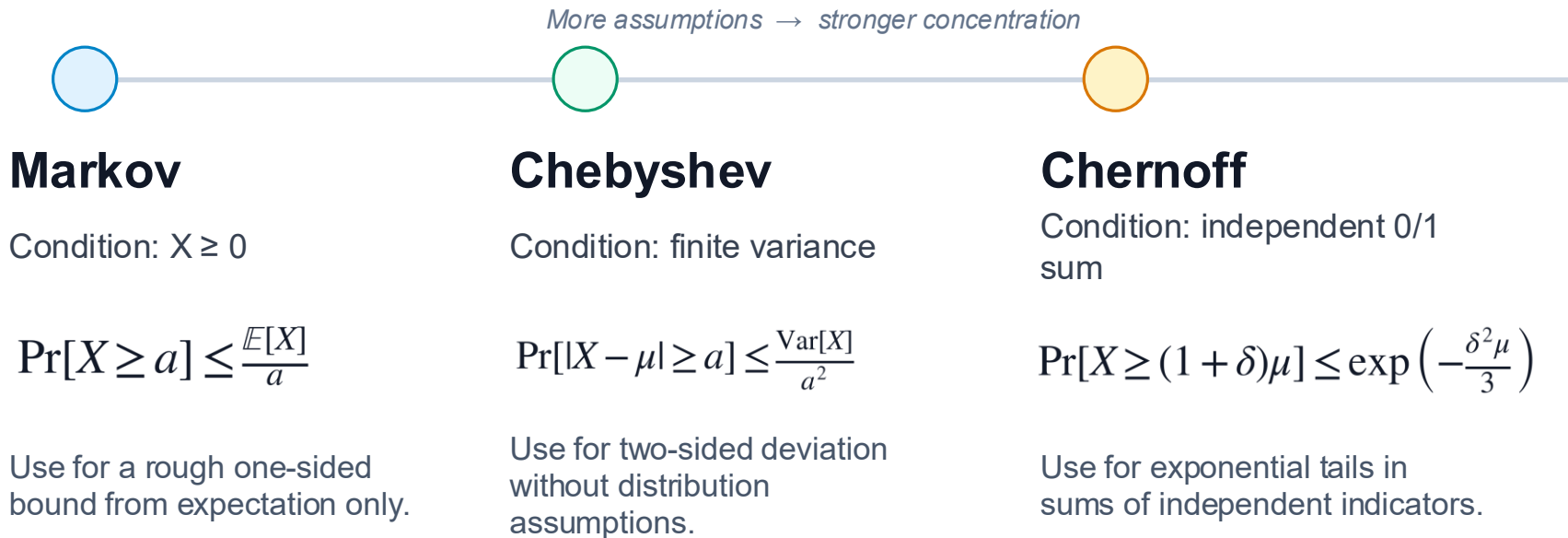
- Choose Φ or credits matching stored future work.
- Show cheap operations deposit $O(1)$ credit.
- Show expensive operation releases enough credit.
- Check $\Phi \geq 0$ and initial $\Phi = 0$.

Randomized analysis

- Define X or bad event explicitly.
- For expectation: use indicators and linearity.
- For failure: write exact probability first.
- Then apply product form / exponential bound.

Tail Bounds & Union Bound

Key conditions, formulas, and when to use them in randomized-algorithm analysis



Union Bound

Condition: no independence needed

$$\Pr\left[\bigcup_i E_i\right] \leq \sum_i \Pr[E_i]$$

Use after proving a failure probability for one fixed object; then extend it to “any object fails.”